

Automated Planning Theory and Practice, Academic Year 2022-2023

Dimitri Vinci, ID 223960

dimitri.vinci@studenti.unitn.it

Abstract

In this report, content and results of Automated Planning Theory and Practice year 2022-20223 project course is exposed. First chapter describes modelling approach and assumptions followed in every exercise. In second chapter experiments conducted over modelling is described. First section of this chapter describes problem definition approach, while following sections show out results. Third chapter report critical discussion of results and conclusions (as long as some possibility of further development). Finally, references are reported.

1. Modelling approach and assumptions

In the following sections modelling approach and choices are discussed as long as content of directories. A schemata of directory content is reported in table 1

Path	Content
~/ATP project - Dimitri Vinci - 223960	Contains report and code directory
~/ATP project - Dimitri Vinci – 223960/Project_code	Contains all project code divided in subdirectory and proofs of executions
~/ATP project - Dimitri Vinci – 223960/Project_code/ex1_ex2	Contains code and proof of executions for exercise 1 and exercise 2
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1	Contains code and proof of executions for exercise 1
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/box_assumption	Contains code and proof of executions for exercise 2, with assumption of number of box equal number of item to be delivered
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/box_assumption/strip_plus *	Contains code and proof of executions for exercise 2, with assumption of number of box equal number of item to be delivered, strip implementation
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/box_assumption/strip_plus_neg *	Contains code and proof of executions for exercise 2, with assumption of number of box equal number of item to be delivered, strip + negative precondition implementation
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/box_assumption/adl_plus *	Contains code and proof of executions for exercise 2, with assumption of number of box equal number of item to be delivered, adl implementation
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/box_assumption/adl_plus_neg *	Contains code and proof of executions for exercise 2, with assumption of number of box equal number of item to be delivered, adl + negative precondition implementation
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/no_box_assumption	Contains code and proof of executions for exercise 2, without assumption of number of box equal number of item to be delivered
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/no_box_assumption/strip_plus *	Contains code and proof of executions for exercise 2, without assumption of number of box equal number of item to be delivered, strip implementation
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/no_box_assumption/strip_plus_neg *	Contains code and proof of executions for exercise 2, without assumption of number of box equal number of item to be delivered, strip + negative precondition implementation
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/no_box_assumption/adl_plus *	Contains code and proof of executions for exercise 2, without assumption of number of box equal number of item to be delivered, adl implementation
~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1/no_box_assumption/adl_plus_neg *	Contains code and proof of executions for exercise 2, without assumption of number of box equal number of item to be delivered, adl + negative precondition implementation

~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2	Contains code and proof of executions for exercise 2
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/num_version	Contains code and proof of executions for exercise 2, with numeric fluents
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/num_version/strip_plus *	Contains code and proof of executions for exercise 2, with numeric fluents, strip version
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/num_version/strip_plus_neg *	Contains code and proof of executions for exercise 2, with numeric fluents, strip version + negative preconditions
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/num_version/adl_plus *	Contains code and proof of executions for exercise 2, with numeric fluents, adl version
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/num_version/adl_plus_neg *	Contains code and proof of executions for exercise 2, with numeric fluents, adl version with negative precondition
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/non_num_version *	Contains code and proof of executions for exercise 2, without numeric fluents
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/non_num_version/strip_plus *	Contains code and proof of executions for exercise 2, without numeric fluents, strip version
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/non_num_version/strip_plus_neg *	Contains code and proof of executions for exercise 2, without numeric fluents, strip version + negative precondition
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/non_num_version/adl_plus *	Contains code and proof of executions for exercise 2, without numeric fluents, adl version
~/ATP project - Dimitri Vinci – 223960/project_code/ex1_ex2/ex2/non_num_version/adl_plus_neg *	Contains code and proof of executions for exercise 2, without numeric fluents, adl version + negative precondition
~/ATP project - Dimitri Vinci – 223960/project_code/ex3	Contains code and proof of execution for exercise 3
~/ATP project - Dimitri Vinci – 223960/project_code/ex3/ex3_parallel *	Code and proof of execution for parallel domain implementation for exercise 3
~/ATP project - Dimitri Vinci – 223960/project_code/ex3/ex3_sequential *	Code and proof of execution for sequential domain implementation for exercise 3
~/ATP project - Dimitri Vinci – 223960/project_code/ex4_ex5	Contains code and proofs of execution for exercise 4 and exercise 5
~/ATP project - Dimitri Vinci – 223960/project_code/ex4_ex5/ex4	Contains ex4 implementations and its proof of execution
~/ATP project - Dimitri Vinci – 223960/project_code/ex4_ex5/ex4/non_num_version *	Contains ex4 implementations for non numeric fluents domain implementation and its proof of execution
~/ATP project - Dimitri Vinci – 223960/project_code/ex4_ex5/ex4/num_version *	Contains ex4 implementations for numeric fluents domain implementation and its proof of execution
~/ATP project - Dimitri Vinci – 223960/project_code/ex4_ex5/ex5	Contains plansys2 implementation and its proof of execution

* this directory also contains problem file and proof of concepts

Table 1: directory content description

1.1. Common modelling approach and assumptions

Here is reported a brief analysis of design choices and assumptions valid for all exercises. Exercises specific assumptions are described thereafter. Quoting from problem description:

"some people might not need food, medicine, or tool; " "some people might need both food and medicine, or food and tool, or the three of them, and so on ". This could be faced via typed person like:

```
Person - object ;; generic person
person_m - person ;; a person who want medicine
person_f - person ;; a person who want food
person_mf - person;; a person who want medicine and food
...
```

Note however that this implies defining specific actions to deliver combinations of items (methods whose typing conditions are based on general person type are also not possible if one want define a person which do not want anything, due to type inheritance. This situation is not tested in this work but could be considered in real setting or evolution of domain at hand). In present case instead author used and/or combination of goals, e.g.

```
:goal (and (or (assigned person1 food1) (assigned person1 food2))
```

This allows to not decide a priori a specific content to be delivered from the contents available, not assigning more than one content (*"Each person either has or does not have a box with a specific content. That is, you must keep track of whether they have food or not, whether they have medicine or not, whether they have tools or not, and so on. "*) and use generic actions. Note that temporal planners do not allow for or conditions in goal so another technique involving predicate was used (see section 1.5).

"Since we want to be able to expand this domain for multiple robotic agents in the future, we expect to use a separate type for robotic agents". This sentence explicitly advocates for a robot type. Not only, in the present formulation, following predicates

```
(is_assigned_box ?b - box)
(not_is_assigned_box ?b - box)
```

Or similarly forall conditions if requirements allow it

```
(forall (?r1 - robot)(not(assigned_robot_box ?b ?r1)))
```

To check if a box is carried by others robots. Without using this approach a robot could simply empty or pick up a box carried by other robots. This will allow to increase number of robots easily simply changing problem files.

"The robotic agents can move directly between arbitrary locations (there is no "roadmap" with specific connections that must be taken), so the graph is fully connected" This was handled simply defining a location type. Being a fully connected graph, it is not necessary to define a predicate to handle reachability between locations

"... if the box is empty...". This make us propend toward setting maximum box capacity to one (this imply existence of a predicate to check if a box is empty). Defining box as type is also suggested by *"... fill a box with a content..."* which implies that robot have to interact with boxes modifying its state. Note that formulation does not specify if there could be boxes with a content already present (at the start of problem). However even if this situation is not considered in problem files employed (see section 2) this is not a fault condition for present solution given existence of predicate to check if a box is already empty before filling a box

"Box contents shall be modeled in a generic way, so that new contents can easily be introduced in the problem instance" Author used object type and subtype given type could be easily modified to extend it with more subtypes:

```
content - object
medicine - content
tool - content
food - content
```

Predicates could not be of easy of usage due to the fact that a predicate do not show inheritance not allowing to use same generic methods for deal with specific content assigned to boxes (e.g. gilling a box would require three actions, one for each content type). Note that using that approach also content objects must be defined in problem files (so contents are not infinite in number, this is an assumption).

"each injured person is at a specific location, and does not move" *"each box is initially at a specific location"*. This advocates for necessity to track position of box and persons with suitable predicates. Note that having defined content as object also a content location predicate is needed.

"each person either has or does not have a box with a specific content". Seem to say that boxes (and not content) is assigned to person. However *"...the people at the same location to have content..."* make us propend toward assigning content not box (this is also confirmed by saying it is necessary to model filling and empty a box).

"empty a box by leaving the content to the current location and causing the people at the same location to have the content". This seem to imply that if a box is emptied, all people in the location where the box is located will have the content. However this could also be interpreted as more general statement that action of empty a box deliver its content toward a single peron in this location (which is also confirmed by *"deliver a box to a specific person who is at the same location"*). Here this last assumption is done to make domain more generic.

"move to another location moving the loaded box". This imply defining different methods if robot (or carrier) is loaded. See section 1.3 to see how this situation is handled with carrier capacity.

"deliver a box to a specific person who is at the same location". This seems to imply that robot could deliver or drop a box only in place where a person is located. This is called by author "box assumption" (see next section for a brief discussion). However, this would limit problems to ones in which boxes could not be reused (since a robot has to drop a box to deliver it, but to reuse it, robot has to take it back to depot where another content is located). So, this assumption is refused to make domain more generic.

PDDL allows for different possible domain requirements. In the table 2 is summarized different naming conventions for domain (and corresponding problem files) as long with different PDDL extension employed given name

Requirements included	name	Pddl level / specific extension involved
:strips	strips_plus	PDDL 1.0
:typing		strips + typing + equality
:equality		
:conditional-effects	adl_plus	PDDL 1.0
:existential-preconditions		
:disjunctive-preconditions		adl + typing + equality
:universal-preconditions		
:strips	strips_plus_neg	PDDL 2.1/2.2
:typing		strips + typing + equality +
:equality		derived predicate + axiom
:negative-preconditions		
:conditional-effects	adl_plus_neg	PDDL 2.1/2.2
:existential-preconditions		
:disjunctive-preconditions		
:universal-preconditions		
:negative-preconditions		adl + typing + equality + negative precondition + derived predicate + axiom

Table 2: naming conventions applied in this work

Notice that not all these extensions were applied in all exercises (see next sections for details) and that exercise 3 uses different naming conventions. Notice that in this work is not considered

- ucpop, safety-constraints and domain-axioms of PDDL 1.0 given is not supported by any planner considered here

- PDDL 3.0 constraints and preferences given are not supported by any planner considered here

Finally notice that

- Names use alphanumerical plus _ characters since are generally accepted [5]
- All predicates/functions/actions etc. accept parameters in alphabetical order of their name.
- Use single ; for single line comments, ;; for inline comments
- Extends and constants domain file keywords are not employed since are not always supported by all planners.

1.2. Exercise 1 / Classical planning approach and assumptions

This exercise is in ~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex1 path. Inside it two subdirectory points toward two modelling versions

- no_box_assumption, allow for less boxes than number of items to be delivered

- box_assumption, suppose number of boxes is same of number of objects. This allows specifying precondition to not return with a box to the depot, theoretically reducing symmetry of the planning solutions (or possible solutions)

For each of these folders, there is a domain and relative problem files for each of the four possible language extensions described in table 2.

1.3. Exercise 2 / Numerical planning approach and assumptions

This exercise is located in ~/ATP project - Dimitri Vinci – 223960/Project_code/ ex1_ex2/ex2 path. Inside it two subdirectory points toward two modelling versions

- Num_version, domain and problem files using numeric fluents to handle capacity.

- Non_num_version, domain and problem files without numeric fluents. Here a predicate for each possible capacity is defined to handle lack of numeric fluents.

In this exercise some considerations are needed. Quoting from problem description: *"though a single carrier is needed for the single robotic agent, there should still be a separate type for carriers; "* This clearly advocates for a carrier object. This design choice implies conversely necessity to define a predicate to assign robot to a carrier. A predicate to check for empty boxes is no more needed (since it is the carrier which carries boxes) and pick up a box now must consider carrier capacity. Note that carriers position could not be handled given it is sufficient to consider reciprocal robot/ robot assigned carrier box positions. There could be possibility to consider capability of robot to pick up (or drop or empty) more than one box at a time but this is not considered since author suppose robot have a single manipulator (this seem also be confirmed by problem description in temporal setting which states that a robot could not pick up or drop a box simultaneously and this would make sense only if robot has a single manipulator).

"...does not have to return to the depot until after all boxes on the carrier have been delivered " This seem to state that robot could not return to depot if all boxes are delivered. However, if small number of boxes must be handled is necessary for the robot to manipulate empty boxes to take it to depot to be reused. So, in this case author added different move actions based on proportion of empty/full boxes. If no box or all empty boxes are carried, then robot could return to depot. To differentiate between a location and the depot this type hierarchy is introduced

```
Generic_location - object
Location depot - generic_location
```

Note that these formulations (as long as ones of previous exercise) include inequality prerequisite in actions to avoid move actions with same boxes or move between same locations, however this is not necessary because

- If more boxes in move actions are the same, having same destination, this does not change effects of the action
- If start and end locations are the same simply the action effect do not change positions of robot and boxes carried

So given this prerequisites are redundant and :equality is not supported by all planners this preconditions are removed in temporal planning.

1.4. Exercise 3 / Hierarchical task network approach and assumptions

This exercise is located in ~/APTP project - Dimitri Vinci – 223960/project_code/ex3. Following IPCC hierarchical task network competition [7] this work consider two possibility (each of them has a separate folder)

- Sequential
a robot fetch and deliver a box one a time.
- Parallel
a robot can take more than one box before deliver to more than one person located in different positions. Note that in this case only a maximum of three deliver in parallel could be handled due to memory error (which is reported in proof of execution folder of parallel task)

All these domains are implemented in totally and partially ordered implementation (however partially ordered could not be executed due to memory constraints but is parsable and so left in directory). Naming conventions for these files are reported in table 3

name	Total vs partial	Sequential vs parallel
T_NR_S	total	sequential
T_NR_P	total	parallel
P_NR_S	partial	sequential
P_NR_P	Partial	parallel

Table 3: naming conventions for hierarchical task network exercise

Coherently with :typing and :hierarchie requirements supported by Panda planner, here strips_plus setting from non_numeric exercise 2 action implementation is used.

1.5. Exercise 4 / Temporal planning approach and assumptions

This exercise is located in ~/ATP project - Dimitri Vinci – 223960/project_code/ex4_ex5/ex4. Inside it two subdirectory contains two different implementation possibilities

- Num_version, domain and problem files using numeric fluents to handle capacity
- Non_num_version, domain and problem files without numeric fluents. Here a predicate for each possible capacity is defined to handle lack of numeric fluents.

Note that only strips_plus setting (as reported in table 2) is considered here due to requirements restriction of Optic and Popf (which do not allow for existential and negative preconditions). Notice also that quoting the text:

"Choose arbitrary (but reasonable durations) for the different actions. "

Being general it is supposed that following time durations constraints hold

- fill_box, empty, deliver, pick_up have duration lower than movements action
- movement actions have duration higher than all other actions

In this work is supposed that all action in first action set have same duration which is selected to be 1, and similarly 2 for movement action set. Note that this choice is not relevant given there are no concurrent actions (see later) and that this time value could be changed arbitrarily by end user.

"Consider the possibility to have actions to be executed in parallel when this would be possible in reality. For example, a robotic agent cannot pick up several boxes at the same time, or pick up a box and fly to a destination at the same time. "

Note that having a single actuator, and not being capable of act during movements in the present domain this is equivalent to force nonconcurrency. This could be done via a shared resource for each actions which depends on the robot involved in the action (practically using a predicate robot_occupied ?r – robot). This predicate is set to true at start of the preconditions and than set to false at start of effects, and true at end of effects [5]. Note finally that this planner do not allows for or in :goal problem definitions so a different approach is used to deliver item in domain file. Each person has a related predicate to express necessity to have delivered a specific type of content. This predicate specific deliver is handled via three version of empty action (one for each type of content).

1.6. Exercise 5 / Plansys2 approach and assumptions

In this final exercise strip_plus temporal non numeric fluent version is employed. The remaining content of folder follow model of original example with C action implementation for every action available in the original pddl domain. A command example is inserted in launch directory following first problem file of exercise 4.

2. Experiments

2.1 Problem file constructions

To develop different problem files, inspiration is taken by similar problems used in IPCC (a full list of IPCC competitions is available at [6]). In particular, domain considered here to been taken as references are reported in table 4

Name	Employed in	Characteristics	Reference links
Gripper	IPCC 1998	deterministic non-numerical non-temporal	https://ipc98.icaps-conference.org/
Logistic world	IPCC 1998 IPCC 2000	Deterministic non-numerical non-temporal	https://ipc00.icaps-conference.org/ https://ipc98.icaps-conference.org/
Miconic	IPCC 2011	deterministic temporal/non-temporal non-numerical	http://www.plg.inf.uc3m.es/ipc2011-deterministic/
Driverlog	IPCC 2002	deterministic numerical/non-numerical temporal/non-temporal	https://ipc02.icaps-conference.org/
Zenotravel	IPCC 2002	deterministic temporal/non-temporal non-umerical/numerical	https://ipc02.icaps-conference.org/
Satellite	IPCC 2004	deterministic numerical/non-numerical non-temporal	https://ipc04.icaps-conference.org/deterministic/

Settlers	IPCC 2002	IPCC 2002: deterministic numerical non-temporal	https://ipc02.icaps-conference.org/
	IPCC 2004	IPCC 2004: deterministic non-temporal/temporal numerical/non-numerical	https://ipc04.icaps-conference.org/deterministic/
Trucks	IPCC 2006	temporal/non-temporal non-numerical deterministic	https://ipc06.icaps-conference.org/deterministic/
Storage	IPCC 2006	temporal/non-temporal deterministic	https://ipc06.icaps-conference.org/deterministic/
Tidybot	IPCC 2011	deterministic numerical/non-numerical non-temporal/temporal	
Transports	IPCC 2011	Non-temporal Non-numeric deterministic	https://ipc06.icaps-conference.org/deterministic/
Rovers	IPCC 2002	deterministic numerical	https://ipc02.icaps-conference.org/
	IPCC 2004	temporal/non-temporal	https://ipc04.icaps-conference.org/deterministic/

Table 4: fons of inspirations for problem files

Following observation could be done on IPCC problems file:

- In general IPCC use 20 different domains to test performances of planners.
- There is no strict coherence if enumeration follow increasing on difficulty or decreasing of difficulty. For example, in Driverlog first number of packages to be delivered is increased (increased complexity) than number of people which packages are delivered is decreased (reducing complexity).
- Typically, IPCC follow this schema: increase intrinsic difficulty due to number of goals to be satisfied, then given number of goal to be satisfied generate problem file with increasing structural difficulty meaning difficulty due for example more complex map in routing problems as in zenotravel, miconic, transports etc. This approach is followed by logistic, miconic, zenotravel, transports, driverlog
- There is no specific enumeration or approach differences in elaborating problem files between numerical/temporal/classical problems.

given this observation it will be used in general

- 20 difficulty increasing problems. Notice that even if 20 problems were elaborated it was not possible (for memory and time constraints) to execute it all in all settings, this will be reported thereafter.
- here first structural difficulty is increased than number of goals to be satisfied.
- there will be no differences in how change complexity between numerical/classical/temporal.

The problem modelled in this work

- It has some degree of goals interaction (especially in numeric setting in which capacity could be exploited to satisfy more subjects with less step, and in general due to position of people which could be satisfied more easily if less number of squares are involved due to less movements).
However different proportion in delivered items do not influence complexity given they are delivered in exact same way (an opposite example is truck problem that had to deal with refrigerating goods that have to be delivered via specialized trucks). the only constraints involved in goals is due to type of content to be delivered (there are no other goal requirements that could influence complexity)
- there is no structural complexity due to routing problems (however as said before more square influence complexity)

given these observations, the following measurements of problem difficulty are defined

- number of requests
it influences number of necessary steps being directly connected to goal satisfaction. Here NRT number of requests per type is defined. Notice this metric is computed "per type" because different object types do not involve different difficulties in the problem. Here is defined maximum number of objects per type to be delivered as 4 (note that maximum complexity in this case is reached when 12 people located in different squares require one type specific item). Given the maximum number of objects per type defined.

$$NNRT = \text{normalized number of requests} = \frac{NRT}{4}$$

note that this metric is globally normalized in the sense that it has value between 1 to 0 between all 20 problems proposed.

- object request

A greater number of items that could be used to be delivered than number of objects to be delivered allow for more possibility to satisfy the problem increasing symmetry of problem meaning there are more way to reach the goal and more possibility to satisfy actions prerequisite which could increase difficulty in find solution. In present case, maximum number of items is set to double the maximum request per type of item for the problem at hand. So the following two metrics are defined

$$OR = \text{object request} = \frac{\text{total number of item requested}}{\text{total number of item available}}$$

$$NOR = \text{negative object requests} = 1 - OR$$

- box exploitation

lower number of boxes influence number of travel to satisfy a person

$$BE = \text{box exploitation} = \frac{\text{total number of item requested}}{\text{number of boxes}}$$

$$NBE = \text{normalized box exploitation} = \frac{BE}{\max(BE)} = \frac{BE}{\max(BE)} * (\text{total number of item requested})$$

$$= \frac{1}{\text{number of boxes}}$$

- square occupation

a greater number of people in same square/position reduce number of action necessary to be satisfied (if capacity greater than 1 is allowed for carrier)

$$SO = \text{square occupation} = \frac{\text{number of persons}}{\text{number of squares/position}}$$

$$NNSO = \text{negative normalized square occupation} = \max(SO) - \frac{SO}{\text{number of persons}}$$

- preference distribution

more preferences for a single person reduce number of steps to be necessary for satisfy a goal (given total number of goal to be satisfied and carry capacity greater than 1) because is equivalent to the case in which two people with two different preferences are occupy same square / position

$$NPP = \text{normalized preference per person} = \frac{\text{number of items to be delivered per type}}{\text{total number of people}}$$

$$ONPP = \text{opposite normalized preference per person} = 1 - NPP$$

Given previous metrics a TPDM (total problem difficulty measure) could be defined

$$TPDM = w_1 * NNR + \left(\frac{1-w_1}{4}\right) * (OR + NBE + NNSO + ONPP)$$

Where w_1 is discretionary set to 0.7 to represents higher influence of number of goals to number of steps necessary to satisfy a problem instance. Notice that also $\left(\frac{1-w_1}{4}\right)$ give discretionary same weight to all other metrics which are involved in these total measures which represents structural complexity of the problem at hand.

In table 5 is then presented characteristics of problem files employed to test model file developed in this work

		Problem details					number of requests		object request		box		preference distribution		
Progressivo	Number of available items per type	Total number of available items	Number of total requests	Número of boxes	Number of square position	Number of persons	NRT	NNRT	NOR	NBE	SO	NNSO	NPP	ONPP	TPDM
1	1	3	3	3	1	1	1	0,25	0	0,333333333	1	0	1	0	0,025
2	1	3	3	3	1	2	1	0,25	0	0,333333333	2	0	0,5	0,5	0,0625
3	1	3	3	2	1	2	1	0,25	0	0,5	2	0	0,5	0,5	0,075
4	2	6	3	2	2	2	1	0,25	0,5	0,5	1	0,5	0,5	0,5	0,5
5	2	6	3	1	2	3	1	0,25	0,5	1	1,5	0,5	0,333333333	0,666666667	0,55
6	2	6	6	6	1	2	2	0,5	0	0,166666667	2	0	1	0	0,0125
7	3	9	6	6	2	2	2	0,5	0,333333333	0,166666667	1	0,5	1	0	0,308333333
8	3	9	6	5	3	4	2	0,5	0,333333333	0,2	1,333333333	0,666666667	0,5	0,5	0,360833333
9	4	12	6	4	5	4	2	0,5	0,5	0,25	0,8	0,8	0,5	0,5	0,50375
10	4	12	6	3	6	6	2	0,5	0,5	0,333333333	1	0,833333333	0,333333333	0,666666667	0,525
11	3	9	9	9	1	3	3	0,75	0	0,111111111	3	0	1	0	0,008333333
12	3	9	9	7	3	5	3	0,75	0	0,142857143	1,666666667	0,666666667	0,6	0,4	0,090714286
13	5	15	9	7	5	5	3	0,75	0,4	0,142857143	1	0,8	0,6	0,4	0,410714286
14	5	15	9	5	7	7	3	0,75	0,4	0,2	1	0,857142857	0,428571429	0,571428571	0,432142857
15	6	18	9	5	7	9	3	0,75	0,5	0,2	1,285714286	0,857142857	0,333333333	0,666666667	0,516785714
16	4	12	12	12	1	4	4	1	0	0,083333333	4	0	1	0	0,00625
17	4	12	12	8	4	4	4	1	0	0,125	1	0,75	1	0	0,065625
18	6	18	12	8	7	9	4	1	0,333333333	0,125	1,285714286	0,857142857	0,444444444	0,555555556	0,373660714
19	6	18	12	6	9	9	4	1	0,333333333	0,166666667	1	0,888888889	0,444444444	0,555555556	0,379166667
20	8	24	12	6	9	12	4	1	0,5	0,166666667	1,333333333	0,888888889	0,333333333	0,666666667	0,516666667

Table 5.: description of problem files

2.2 Experiments results

In the following sections details about experiment results are exposed as long with planner and related parameters employed, divided by exercise. To reduce randomness in experiments, each run was repeated 15 times. However only for memory and planner time consistent variation in results were noticed (this is due to usage of deterministic steps in planners and domains which are small and not allow for many solutions) so only in this cases performance results metrics are than averaged and also standard deviation is reported while in other cases only results is reported. Experiments reports following metrics.

- solution length
as often reported as main evaluation parameter (e.g. [14][1]).
- total planner time (s)
some planner works (e.g. [1][15]) consider also time for solution, in present case given uncontrolled setting , author preferred to report total planner time if planner give this metrics as output
- planner memory (KB)
if planner provide it, also total memory employed is reported.

Finally, where more than one planner were employed, following IPCC satisficing track [8] performance measure ratio C^*/C where C is the cost of the cheapest discovered plan for planner and C^* is the cost of a reference plan (which is shortest plan founded by all planner used for this section and for specific problem) is reported . The score of a planner is the sum of its scores for all tasks. Experiments were run on x86_64 Architecture, 64 bit, 8 cpu, Intel(R) Core(TM) i7-6700HQ CPU @ 2.60GHz, on Linux machine Ubuntu.

2.2.1. Exercise 1 / Classical planning experiments results

Here planutils available planners are considered. In particular FastForward [4] with default parameters and FastDownward [3] in lama-first configuration (which is faster than other IPCC configurations due it searches for only a single heuristic [2]) is employed. Both planners support different domains provided for modelling the problem (fastforward support [4], fastdownward support [11]).

The figure 1 reports results collected for (respectively) FastForward and FastDownward.. The performance of fast downward is 37.40, while performance of fast forward is 35.03.

2.2.2. Exercise 2 / Numerical planning experiments results

Here for non-numerical version of exercise 2, fast forward and fast downward as before are employed. For the numeric version of this setting metric-fast forward [9] with -s 0 (meaning no solution optimization) is used (this because of being an extension of fast forward it supports same requirements). Note that time and memory requirements restrict us to test it only on first 6 developed domains. The figure 2 reports results collected for (respectively) FastForward , FastDownward. And metric-FastForward

2.2.3. Exercise 3 / Hierarchical task network experiments results

Here Panda planner [11] in its default configuration is employed (with -seed 0 option set) The figure 3 reports pandas planner results for sequential and parallel , totally order version. time and memory requirements restrict us to test it only on first 5 developed domains.

2.2.4. Exercise 4 / Temporal planning experiments results

Herre both Optic [10] and popf [13] planner are employed (where second one is considered because default of Ros2 planning system). The figure 4 reports results collected for (respectively) optic and popf planner. Time and memory requirements restrict us to test it only on first 5 developed domains.

Fast Forward									
strip plus					strip plus neg				
progressive number	total time (avg)	total time (std)	number of steps	performance ratio (avg)	total time (avg)	total time (std)	number of steps	performance ratio (avg)	total time (std)
1 box ass	0	0	16	1	0	16	1	0	16
2	0	0	16	1	0	16	1	0	16
2 box ass	0	0	16	1	0	16	1	0	16
3	0	0	18	1	0	18	1	0	18
4	0	0	21	1	0	21	1	0	21
5	0	0	21	1	0	21	1	0	21
6	0	0	45	1	0	45	1	0	45
7	0.019686875	0.0001003035	35	1	0.0197395301	0.003616422	35	1	0.0197395301
7 box ass	0.023665913	0.000970661	35	1	0.023665913	0.000970661	35	1	0.023665913
8	0.009576192	0.000726642	40	0.925	0.009576192	0.000726642	40	0.925	0.009576192
9	0.324091317	0.063437717	42	0.928571429	0.324091317	0.063437717	42	0.928571429	0.324091317
10	0.33959778	0.10498875	43	0.318994682	0.33959778	0.10498875	43	0.318994682	0.33959778

downward (LAMA-tris)									
strip plus neg					adl plus				
progressive number	total time (avg)	total time (std)	number of steps	performance ratio (avg)	total time (avg)	total time (std)	number of steps	performance ratio (avg)	total time (std)
1 box ass	0	0	16	1	0	16	1	0	16
2	0	0	16	1	0	16	1	0	16
2 box ass	0	0	16	1	0	16	1	0	16
3	0	0	18	1	0	18	1	0	18
4	0	0	21	1	0	21	1	0	21
5	0	0	21	1	0	21	1	0	21
6	0	0	45	1	0	45	1	0	45
7	0.019686875	0.0001003035	35	1	0.0197395301	0.003616422	35	1	0.0197395301
7 box ass	0.023665913	0.000970661	35	1	0.023665913	0.000970661	35	1	0.023665913
8	0.009576192	0.000726642	40	0.925	0.009576192	0.000726642	40	0.925	0.009576192
9	0.324091317	0.063437717	42	0.928571429	0.324091317	0.063437717	42	0.928571429	0.324091317
10	0.33959778	0.10498875	43	0.318994682	0.33959778	0.10498875	43	0.318994682	0.33959778

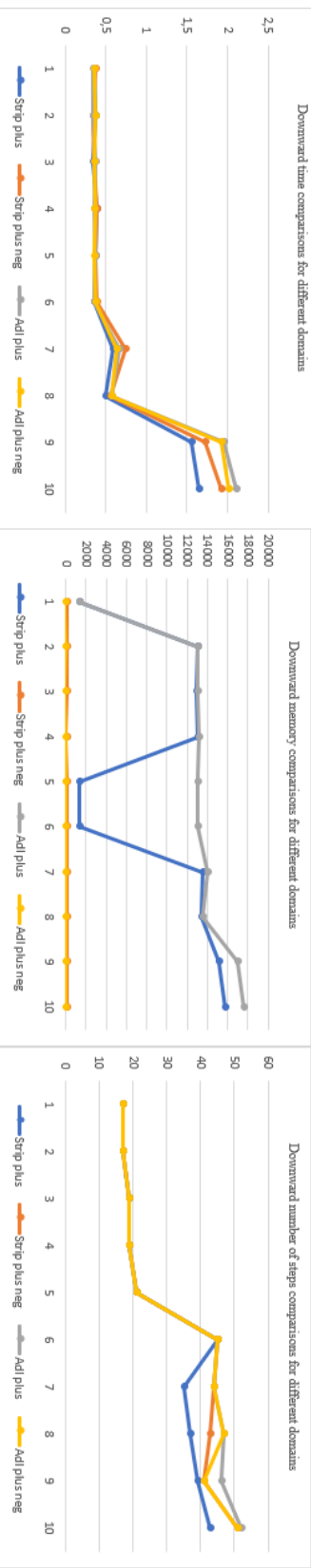


Figure 1: results and graphs for exercise 1

downward (LMA-first)											
strip plus				strip plus neg				adl plus			
total time (avg)	total time (std)	memory (avg)	memory (std)	number of steps	performance (avg)	total time (std)	total time (std)	memory (avg)	memory (std)	number of steps	performance (avg)
17.49591265	0.239416223	13721.9	68	16	0.668686667	1.767370338	0.542380578	14230.9	331.9537162	18	0.668686667
17.96023963	0.176535012	13636.4	69.6162464	16	0.668686667	0.863260144	0.4230.9	175.4777916	14230.9	18	0.668686667
0.92614348	0.018626347	13812.4	70.8872001	17	0.541776471	0.538265675	0.3007.0	17.9637877	50.9837877	17	0.541776471
0.92614348	0.018626347	13812.4	70.8872001	21	0.3	7.93952897	0.063321121	138002.4	60.51298278	22	0.87687618
0.94639122	0.044010021	12946.2	234.956339	21	0.919052897	0.063321121	0.028033382	12975.9	18.9562784	21	0.87687618
0.65773254	0.02274016	13865.7	234.088809	45	1	0.63965344	0.028033382	12975.9	18.9562784	45	1

Metric-F - s-0

Metric-F - s-0											
strip plus				strip plus neg				adl plus			
total time (avg)	total time (std)	memory (avg)	memory (std)	number of steps	performance (avg)	total time (std)	total time (std)	memory (avg)	memory (std)	number of steps	performance (avg)
1	0	0	0	14	0.857142857	0	0	0	0	16	0.8
2	0	0	0	14	0.857142857	0	0	0	0	16	0.8
3	0	0	0	16	1	0	0	0	0	16	1
4	0	0	0	18	1	0	0	0	0	18	1
5	0	0	0	21	1	0	0	0	0	21	1
6	0.080854578	0.00077851	45	45	1	0.079720411	0.000640719	0.089683486	0.000967966	45	1

Fast Forward											
strip plus				strip plus neg				adl plus			
total time (avg)	total time (std)	memory (avg)	memory (std)	number of steps	performance (avg)	total time (std)	total time (std)	memory (avg)	memory (std)	number of steps	performance (avg)
1	0	0	0	14	0.857142857	0	0	0	0	16	0.8
2	0	0	0	14	0.857142857	0	0	0	0	16	0.8
3	0	0	0	16	1	0	0	0	0	16	1
4	0	0	0	18	1	0	0	0	0	18	1
5	0	0	0	21	1	0	0	0	0	21	1
6	0.14658859	0.0610833280	45	45	1	0.157230236	0.010408135	0.086330348	0.008939888	45	1

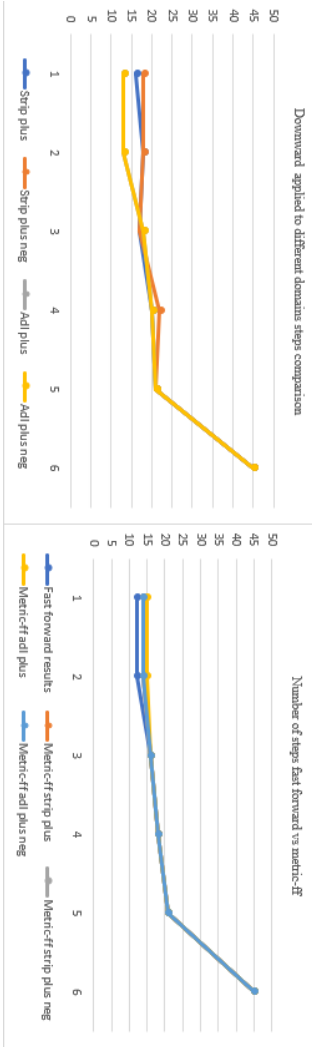
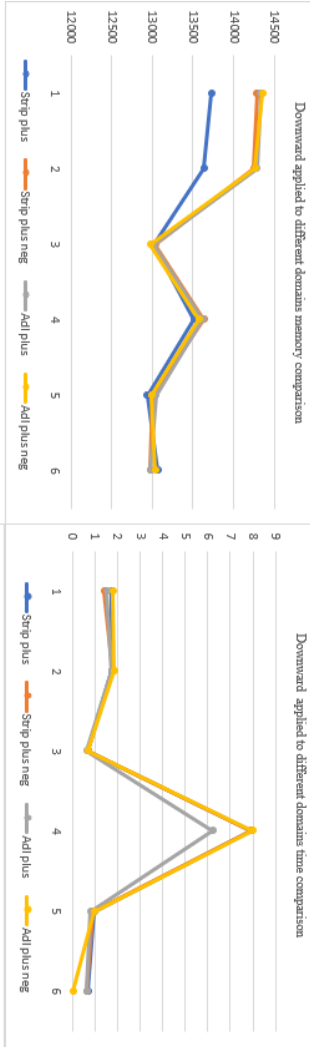


Figure 2: results and graphs for exercise 2

pandas (seed 0)										
progressive number	T-NR-S						T-NR-P			
	total time (avg)	total time (std)	memory (avg)	memory (std)	numPrimTasks	total time (avg)	total time (std)	memory (avg)	memory (std)	number of primitive tasks
1	3,383768537	0,140747552	146343580,5	3622,394121	38	6	0,136098609	277822832,2	2141,656773	47
2	3,385761365	0,178318522	159563419,3	79306,36628	38	6,406093721	0,121725213	264603482,4	2001,533146	47
3	1,875634547	0,146894511	94401463,9	901,8874591	43	2,275093582	0,217487114	99970966,2	7722,411207	50
4	3,877666813	0,124544072	174051341	1359,762398	43	7,584890827	0,151873177	363982530,9	1610,158132	50
5	1,934301321	0,21710036446	85871770,9	1514,912058	48	2,019465443	0,203209586	85353697,4	5659,529156	48

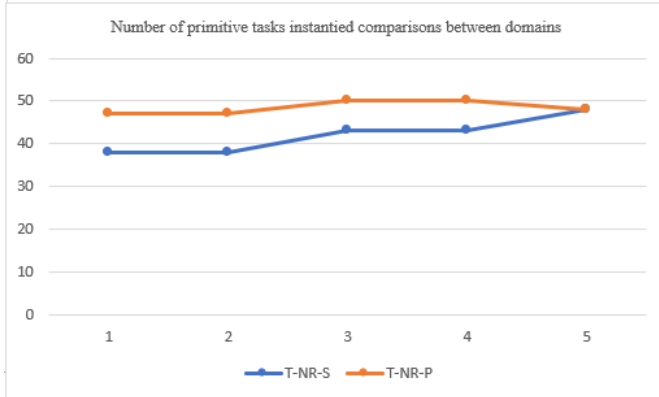
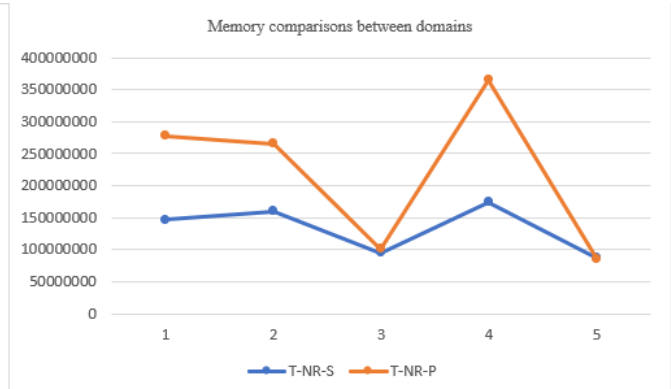
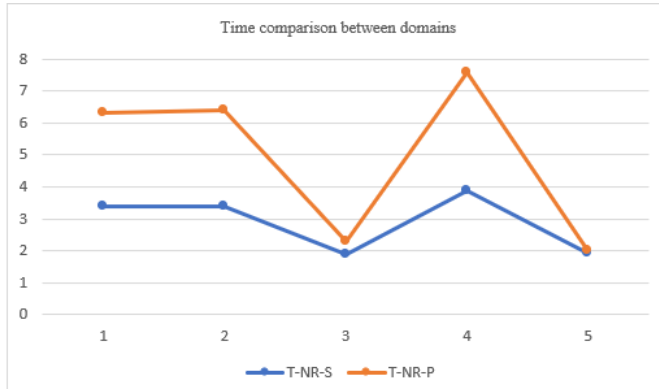


Figure 3: results and graphs for exercise 3

optic -N								
progressive number	strip plus				strip plus w. Numeric			
	total time (avg)	total time (std)	number of steps	plan length	total time (avg)	total time (std)	number of steps	plan length
1	0,30284399	0,009433221	12	14,012	0,3274843	0,010831165	26	34,026
2	0,280567484	0,005652684	12	14,012	0,3320509	0,010130968	26	34,026
3	0,190029257	0,011454729	16	20,016	0,2134958	0,007295758	24	30,024
4	0,490541403	0,011151604	16	20,016	0,512984	0,01169373	20	24,02
5	0,248200792	0,010467349	20	26,02	0,3123405	0,005137012	20	26,02

popf								
progressive number	strip plus				strip plus w. Numeric			
	total time (avg)	total time (std)	number of steps	plan length	total time (avg)	total time (std)	number of steps	plan length
1	0,238760321	0,003596961	12	14,012	0,516664189	0,010697412	26	34,026
2	0,22695545	0,004144586	12	14,012	0,301237336	0,019041531	26	34,026
3	0,1493295045717	0,006667998	16	20,16	0,164505917	0,012260082	24	30,024
4	0,44043329	0,004026669	16	20,016	0,502432079	0,016187609	20	24,02
5	0,239088004	0,0051096	20	26,02	0,273835128	0,013082147	20	26,02

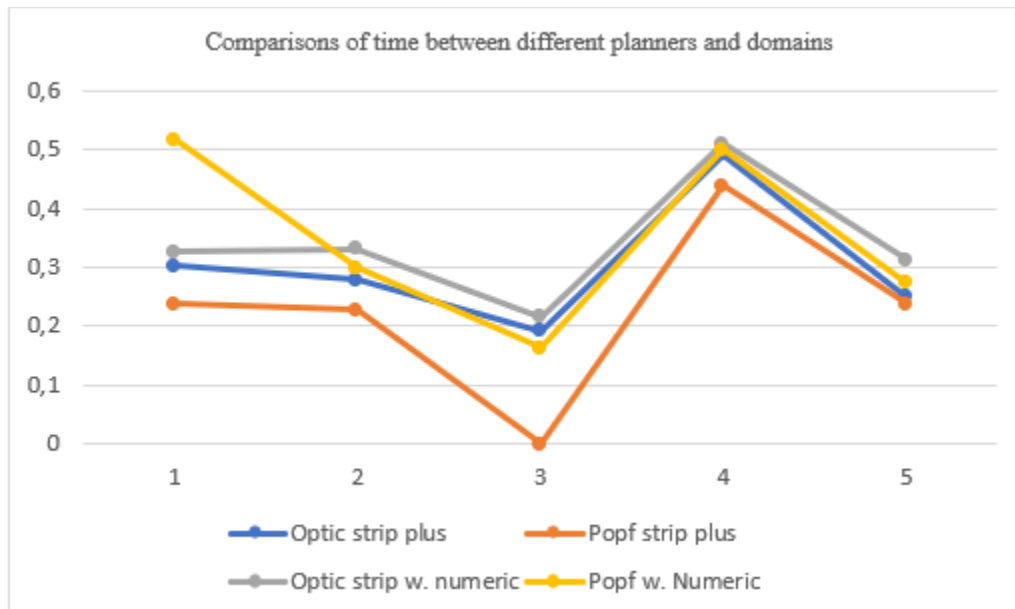


Figure 4: results and graphs for exercise 4

3. Critical discussion and Conclusions

Exercise 1, demonstrates general better “planner performance” (as measured following IPCC satisficing track procedure) of fast downward. Note that this could be expected since fast downward is an improvement of fast forward. Memory and time to solution do not modify in fast downward using different requirements (this could be observed also in exercise 2). This could be due to internal representations (from which fast downward try finding solution) in which all problems are translated by fast downward that are independent of specific formulations. Note also that the same domain transformations could also be responsible for general faster time to solution of fast forward (also in exercise 2) in respect to fast downward. So Fast Downward seems more suitable to obtain optimal or least solution cost plans, while fast forward seem to be better for fast planning.

Interestingly metric-ff and fast forward do not show different time to solutions in exercise 2 (even if the first one use numeric fluent). Probably this is due to the fact the problem is small in dimensions not allowing to show consistent differences in problem execution.

Regarding exercise 3, time and memory involved in parallel implementations of domains is greater than the sequential one. This makes sense since (as could also be seen in number of primitive tasks instantiated columns) the parallel version instantiates more primitive tasks. However parallel actions in which robot does not return to depot every time need satisfy a person, could improve solution when task is not fully connected (so path to depot could be longer than path between person to be rescued), when there are multiple depots and so on.

Regarding exercise 4, non-numeric version seems have less time to plan. This could be due to needing of predicate instantiations of numeric fluents. Both produce same plan (this probably because domains tested are small and plans have low length and low possibility to be optimized). Interestingly popf seems to have smaller time to solution which could arise from the fact that popf applies a simple forward chaining procedure with partial-order planning while optic, being based on popf, applies others heuristic and optimizations.

It would be interesting to use :timed-initial-literals to model situation in which box, content and injured people are added every k timesteps. This would lead to a more realistic online scenario. Notice also that as said before present work implementation allows to transition seamlessly to a multi agent setting which would be interesting to assess.

References

- [1] Automatic Derivation of Memoryless Policies and Finite-State Controllers Using Classical Planners, Bonet, B., Palacios, H., Geffner, H., 2009
- [2] Fast Downward aliases, available at: <https://github.com/aibasel/downward/blob/main/driver/aliases.py>
- [3] Fast Downward, available at: <https://www.fast-downward.org/>
- [4] Fast Forward, available at <https://fai.cs.uni-saarland.de/hoffmann/ff.html>
- [5] Haslum, P., Lipovetzky, N., Magazzeni, D., Muise, C., An Introduction to the Planning Domain Definition Language
- [6] IPCC competitions, available at: <https://www.icaps-conference.org/competitions/>
- [7] IPCC Hierarchical task network competition, year 2020, available at: <https://ipc2020.hierarchical-task.net/>
- [8] IPCC, track descriptions, available at: <https://ipc2018-classical.bitbucket.io/#tracks>
- [9] Metric Fast Forward, available at: <https://fai.cs.uni-saarland.de/hoffmann/metric-ff.html>
- [10] Optic planner, available at: <https://nms.kcl.ac.uk/planning/software/optic.html>
- [11] Panda planner, available at: <https://panda-planner-dev.github.io/>
- [12] Planning Wiki – Fast Downward, available at: <https://planning.wiki/ref/planners/fd>
- [13] Popf planner, available at: <https://nms.kcl.ac.uk/planning/software/popf.html>
- [14] Scalable, Parallel Best-First Search for Optimal Sequential Planning, Kishimoto, A., Fukunaga, A., Botea, A., 2009
- [15] Waking Up a Sleeping Rabbit: On Natural-Language Sentence Generation with FF, Koller, A., Hoffman, J., 2010